

Helix Thready — Token-Bridge Codegen

Table of contents

1. What this is (and the single-source rule)
2. Generated targets & honest consumer status
3. How to run
4. The `--check` drift gate
5. Validation record (2026-07-22, this machine)
6. Closure state of THREADY-DES-LIB-04
7. What is deliberately NOT exported
8. Open items
9. Verified consumer quirks (PenPot 2.17)

Rev	Date	Author	Change
1	2026-07-22	swarm (design · tokens-bridge)	Initial: <code>generate.mjs</code> (no-deps Node parser/emitter for <code>opendesign/tokens.css</code>), seven generated per-platform bindings, <code>--check</code> drift gate, honest validation record. Materializes the generator that <code>design-system.md §7</code> , <code>lipgloss-theme.md §8</code> and <code>platform-map.md §6</code> reference but until now did not exist (<code>[OPEN: THREADY-DES-LIB-04]</code> — see §6)
2	2026-07-22	swarm (design · tokens-bridge)	PenPot defect closure (§9): the real PenPot 2.17 import proved a top-level <code>\$description</code> flips the importer into single-set mode, so <code>web/tokens.json</code> is demoted to the pure W3C DTGG canonical artifact (PenPot-import claim removed) and a new eighth target <code>generated/penpot/tokens.penpot-import.json</code> is emitted natively (multi-set + <code>\$themes</code> , no root <code>\$description</code> , byte-identical to the operator-derived file that actually imported). <code>--check</code> extended with quirk guards + structural diff vs that reference; all target writes now atomic (<code>.tmp</code> + rename). PenPot's silent 71/74 drop of <code>duration / cubicBezier</code> recorded

Table of contents

- 1. What this is (and the single-source rule)
- 2. Generated targets & honest consumer status
- 3. How to run
- 4. The `--check` drift gate
- 5. Validation record (2026-07-22, this machine)
- 6. Closure state of THREADY-DES-LIB-04

- 7. What is deliberately NOT exported
- 8. Open items
- 9. Verified consumer quirks (PenPot 2.17)

1. What this is (and the single-source rule)

`generate.mjs` is the **token-bridge codegen** mandated by `design-system.md §7`: “each non-web platform consumes a **generated** binding, not a hand-kept copy, so a token change propagates.” It aligns with the constitution design mandate (`[CONSTITUTION §11.4.162]` one design system for every surface; `§11.4.6` no invented values — every emitted value is parsed, never restated).

The single source of truth is `../opendesign/tokens.css`. The generator parses its `:root` block (light + structural tokens), its `@media (prefers-color-scheme: dark)` block and its `[data-theme="dark"]`, `.dark` block into one internal model, verifies the two dark blocks agree, and emits every target from that model. Nothing in `generated/` may be edited by hand — every file carries a `GENERATED — DO NOT EDIT` header with the sha256 of the `tokens.css` revision that produced it:

```
sha256(tokens.css) = f0f788d1ee73dc624b2006b597d6fb9ab618190b68bf2425c72edc36f0ca4eaf
```

The Lipgloss (Go) target additionally reads the **normative** mapping tables of `../screens/tui/lipgloss-theme.md §2` (truecolor → ANSI-256 → ANSI-16) and the generator **fails** if that document’s truecolor column ever drifts from `tokens.css` — the doc and the CSS are cross-locked.

Determinism: output ordering follows `tokens.css` declaration order; no timestamps are embedded — regenerating from the same source is byte-identical (that is what makes `--check` a CI gate).

2. Generated targets & honest consumer status

Consumer statuses are carried verbatim from `platform-map.md §2` (verified 2026-07-22 against repo HEADs). A **generated binding is a delivery-ready contract — it is NOT a claim that the consumer app exists or consumes it today.**

Target	File	Format	Intended consumer	Honest consumer status (platform-map §2)
Web / interchange	<code>generated/web/tokens.json</code>	W3C DTCG design tokens — the pure DTCG canonical interchange artifact; color modes as token groups <code>thready-light /</code> <code>thready-dark ;</code> <code>dimension / number /</code> <code>duration / cubicBezier /</code> <code>fontFamily \$type s;</code> aliases as DTCG <code>{references}</code> . NOT the PenPot import file — its top-level <code>\$description</code> flips PenPot 2.17 into single-set mode [VERIFIED 2026-07-22] (§9); PenPot consumers use the <code>penpot/</code> target below	Style-Dictionary-class pipelines; generic DTCG tooling	Web layer itself is PRODUCTION-usable (<code>design_system .ds-*</code>) and consumes the CSS directly — this JSON is for tools, not the Angular app. A direct PenPot 2.17 import of this file was exercised 2026-07-22 and misbehaved (single set “tokens”, 38 colors, no themes — §9), which is why the claim moved to the dedicated target
PenPot import	<code>generated/penpot/tokens.penpot-import.json</code>	PenPot 2.17 design-tokens import projection of the same model: token sets <code>thready-light /</code> <code>thready-dark /</code> <code>thready-structure , \$themes</code> (<code>Thready/Light ,</code> <code>Thready/Dark</code> — UUIDs pinned to the proven import for determinism), <code>\$metadata.tokenSetOrder ;</code> no root \$description and no sha256 header (§9 explains why). ASCII-escaped JSON, byte-identical to the operator-derived file that actually imported (<code>../exports/penpot/tokens.penpot-import.json</code>)	PenPot 2.17 tokens importer	Import exercised 2026-07-22 in PenPot 2.17 : sets + both themes land; 71/74 tokens land — PenPot silently drops <code>duration</code> (×2) and <code>cubicBezier</code> (×1), all three still included (§9)
Compose / KMP	<code>generated/compose/ThreadyColors.kt</code>	Kotlin objects <code>ThreadyColors.LightColors / .DarkColors</code> (<code>Color(0xFF...)</code>), <code>ThreadySpacing /</code>	<code>UI-Components-KMP</code>	Utilities-only scaffold, foreign-branded (<code>Theme.kt</code>) ships a Yole Material-red

Target	File	Format	Intended consumer	Honest consumer status (platform-map §2)
		<code>ThreadyRadius (Dp),</code> <code>ThreadyTypeScale (sp / em), ThreadyMotion .</code> Package <code>digital.vasic.thready.design</code> — the design-system \$7 sample declares no package, so this is <code>[DEFAULT – adjustable]</code>		palette, zero widgets, no CI/publish) <code>[GAP: 8.4]</code> . This file is the contract that replaces the hand-kept palette when THREADY-DES-KMP-01 lands
SwiftUI	<code>generated/swiftui/ThreadyTokens.swift</code>	<code>ThreadyTokens.LightColors / .DarkColors</code> <code>(Color(red: green: blue:) ,</code> 6-decimal floats that round-trip to the source hex), <code>Spacing / Radius / TypeScale (CGFloat)</code>	(none today)	No in-house SwiftUI package exists <code>[OPEN: THREADY-DES-LIB-02]</code> ; the sanctioned iOS path is KMP/Compose. Contract only, in case SwiftUI shims materialize
ArkTS / HarmonyOS	<code>generated/arkts/thready_tokens.ets</code>	<code>ThreadyColorLight / ThreadyColorDark</code> classes (ResourceColor-compatible hex strings), <code>ThreadySpacing / ThreadyRadius (vp), ThreadyTypeScale (fp), ThreadyMotion (ms)</code>	native ArkTS client via <code>helix_shims</code>	<code>helix_shims</code> interface uninspected <code>[GAP: 8.5]</code> <code>[OPEN: THREADY-DES-LIB-03]</code> — contract only
Qt / QML (Aurora)	<code>generated/qml/ThreadyTokens.qml</code>	<code>pragma Singleton</code> <code>QtObject</code> with <code>light / dark</code> sub-objects(<code>color</code> properties) + <code>int / real</code> structure properties	<code>helix_design</code> Qt arm	Verified empty scaffold <code>[GAP: 8.2/8.3]</code> ; Aurora path is native via <code>helix_shims</code> <code>[GAP: 8.5]</code> — contract only
TUI / Lipgloss	<code>generated/lipgloss/thready_palette.go</code>	<code>package theme</code> : dark truecolor vars (\$3 names: <code>Accent , AccentOn , Fg , Muted , BorderColor , BorderHard , Success , Warn , DangerColor , Brand2 , + Brand / Bg / SurfaceWarm)</code> , pinned <code>CompleteColor</code>	Thready TUI (Bubble Tea)	Bubble Tea + Lipgloss pattern VERIFIED in-house <code>(helix_track/llms_verifier/.../tui)</code> ; the Thready TUI itself is not built yet — styles are PROPOSED on the

Target	File	Format	Intended consumer	Honest consumer status (platform-map §2)
		degradation set (§5 option 2) and <code>AdaptiveColor</code> light/dark set (§6) — matching <code>lipgloss-theme.md</code> exactly , with the doc's ASSUMED markers on every ANSI-256/16 pick carried through into the code comments		verified pattern (THREADY-DES-TUI-01)
Flutter	<code>generated/flutter/thready_tokens.dart</code>	<code>ThreadyColorsLight / ThreadyColorsDark</code> (<code>Color(0xFF...)</code> via <code>dart:ui</code>), <code>ThreadySpacing / ThreadyRadius / ThreadyTypeScale</code> (<code>double</code>), <code>ThreadyMotion</code> (<code>int ms</code>)	<code>helix_design</code> Flutter arm	Verified empty scaffold [GAP: 8.2/8.3] — contract only

Token counts per target (colors count both modes; aliases included): `tokens.json` **74** tokens · `tokens.penpot-import.json` **74** tokens (**71** land in PenPot 2.17 — §9) · `ThreadyColors.kt` **71** declarations · `ThreadyTokens.swift` **71** · `thready_tokens.ets` **70** `static readonly` + 1 exported const = **71** · `ThreadyTokens.qml` **73** properties · `thready_palette.go` **39** vars (13 roles × truecolor/Complete/Adaptive) · `thready_tokens.dart` **70** `static const` + 1 top-level const = **71**.

3. How to run

```
# From this directory (no npm install – zero dependencies, Node ≥ 18; node 24 verified):
node generate.mjs          # (re)generates ./generated/** and runs the self-check suite
node generate.mjs --check  # CI drift gate – see §4
```

After any change to `../opendesign/tokens.css` (or to the `lipgloss-theme.md §2` mapping tables), re-run `node generate.mjs` and commit the regenerated outputs together with the source change.

4. The `--check` drift gate

`node generate.mjs --check` re-generates every target into a temp directory and:

1. **Drift:** byte-diffs each fresh output against the committed `generated/**` file (missing or differing file → FAIL, exit ≠ 0).

2. **Hex round-trip:** extracts every color literal from every output (including reconstructing hex from the SwiftUI float triplets) and asserts set-equality with the colors parsed from `tokens.css` — no invented, dropped, or mistyped color can survive.
3. **Structural self-checks:** balanced `{}` `()` `[]` and exact expected symbol counts per target (the stand-in validation for toolchains not present on the machine — see §5).
4. **Source consistency:** the `@media` dark block and the `[data-theme="dark"]`, `.dark` block of `tokens.css` must bind identical values; `lipgloss-theme.md` §2's truecolor column must equal the parsed dark values; both JSON outputs must `JSON.parse`.
5. **PenPot quirk guards (§9):** the `penpot/` target must have **no top-level** `$description` (the single-set trigger), must carry `$themes` (2) + `$metadata.tokenSetOrder` (3), and — while the operator-derived reference `../exports/penpot/tokens.penpot-import.json` exists — must be structurally (parsed-JSON) equal to it; the check reports `byte-identical` when the bytes match too.

Exercised both ways on 2026-07-22 (rev 2): pristine tree → 40/40 PASS, exit 0 (the penpot reference check reporting `byte-identical`); a deliberately tampered hex in `penpot/tokens.penpot-import.json` → `FAIL drift`, exit 1; regenerate → 40/40 PASS again. (Rev 1 exercised the same gate at 32/32 with a tampered `ThreadyColors.kt`.)

5. Validation record (2026-07-22, this machine)

Per the no-bluff bar (`CONVENTIONS.md` §7) — what each output was **actually** validated with. Toolchain availability was probed first: `node` v24.18.0 ✓, `go` 1.26.4 + `gofmt` ✓; `kotlinc`, `swiftc`, `qmlint`, `dart` / `flutter`, `tsc` **MISSING** on this machine.

Target	Validation method	Result
<code>web/tokens.json</code>	<code>JSON.parse</code> (node 24) + hex round-trip + DTCG <code>\$type</code> /alias-reference structure emitted per spec	PASS as DTCG. Direct PenPot 2.17 import exercised 2026-07-22 → single-set misbehavior (the <code>\$description</code> quirk, §9) — this file is deliberately NOT the PenPot import file anymore
<code>penpot/tokens.penpot-import.json</code>	<code>JSON.parse</code> + hex round-trip + no-root- <code>\$description</code> guard + <code>\$themes</code> / <code>\$metadata</code> guard + structural diff against the operator-derived file that actually imported into PenPot 2.17 (2026-07-22)	PASS — byte-identical to the proven import file (<code>sha256 eae6bca3...</code>); the import itself landed sets + 2 themes, 71/74 tokens (§9)
<code>compose/ThreadyColors.kt</code>	<code>kotlin</code> MISSING → structural self-checks: balanced delimiters, 71/71 <code>val</code> declarations, hex round-trip	PASS (structural only — not compiler-verified)
<code>swiftui/ThreadyTokens.swift</code>	<code>swiftc</code> MISSING → structural self-checks incl. float→hex round-trip (6-decimal channels reconstruct the exact source hex)	PASS (structural only — not compiler-verified)
<code>arkts/thready_tokens.ets</code>	No ArkTS toolchain (and no <code>tsc</code>) → structural self-checks: balanced delimiters, 70/70 <code>static readonly</code> , hex round-trip	PASS (structural only — not compiler-verified)
<code>qml/ThreadyTokens.qml</code>	<code>qmlint</code> MISSING (also probed qt6 paths) → structural self-checks: balanced delimiters, 73/73 <code>readonly property</code> , hex round-trip	PASS (structural only — not linter-verified)
<code>lipgloss/thready_palette.go</code>	<code>gofmt -e</code> (parse) + <code>gofmt -l</code> (formatting) + <code>go vet</code> + <code>go build</code> in a temp module against real <code>github.com/charmbracelet/lipgloss v0.13.0</code>	PASS (compiles)
<code>flutter/thready_tokens.dart</code>	<code>dart</code> MISSING → structural self-checks: balanced delimiters, 70/70 <code>static const</code> , hex round-trip	PASS (structural only — not compiler-verified)
generator-level	dark-block consistency; <code>lipgloss-theme.md §2</code> truecolor ↔ <code>tokens.css</code> dark equality; drift gate positive + negative test	PASS

The structural-only rows **MUST** be compiler-verified in the consumer repos' CI when the bindings are wired in (§6) — that is part of the open half of THREADY-DES-LIB-04, not a formality.

6. Closure state of THREADY-DES-LIB-04

`platform-map.md §6` recorded: “the token-bridge codegen (CSS → Lipgloss / `ThreadyColors` / Flutter theme) referenced by every non-web cell does not exist yet.” Honest state after this change:

- **CLOSED half:** the generator now **exists in-repo** (this directory), parses the canonical `tokens.css`, emits all seven bindings deterministically, and ships a CI-able `--check` drift gate.
`lipgloss-theme.md §8`’s “this file specifies its TUI output, it does not claim the generator exists” is superseded for the *generator-existence* part.
- **OPEN half: wiring into the consumer repos remains open.** Nothing consumes these files yet: `UI-Components-KMP` still carries the foreign Yole palette `[GAP: 8.4]`, `helix_design` is an empty scaffold `[GAP: 8.2/8.3]`, the Thready TUI is unbuilt, ArkTS/Qt shims are uninspected `[GAP: 8.5]`. Tracked as THREADY-DES-KMP-01 / -FLUT-01 / -QT-01 / -TUI-01 in `component-library.md §10`; compiler verification of the structural-only targets (§5) lands with that wiring.

7. What is deliberately NOT exported

Mirrors the precedent set by `../figma/figma-variables.json` `_meta.consumption_notes`:

- `--accent-hover`, `--accent-active`, `--elev-raised`, `--focus-ring` — CSS `color-mix()` formulas over `var()`; they have no static value. Native platforms derive hover/active/focus treatments from their own state systems over the exported base colors.
- `--elev-flat` (`none`) and `--elev-ring` (a shadow composite over `var(--border)`) — shadow composites, platform-specific by nature.
- Web-only `--font-*` stacks are exported **only** to the DTCG-shaped targets (`web/tokens.json`, `penpot/tokens.penpot-import.json` — `fontFamily`); native targets load fonts through their own asset pipelines (`design-system.md §4`).
- B-slot aliases (`--fg-2`, `--meta`, `--border-soft`) and `--ds-heart` **are** exported — as real DTCG `{references}` / language-level aliases, preserving the alias semantics of `tokens.css`.

8. Open items

- `[OPEN: THREADY-DES-LIB-04]` — narrowed per §6: generator DONE, consumer wiring OPEN.
- `[OPEN: THREADY-DES-17]` — all ANSI-256/16 indices in the Go palette are ASSUMED nearest-color picks (carried verbatim, with markers, from `lipgloss-theme.md §2`); verify on real terminals.
- `[OPEN: THREADY-DES-02]` — PenPot bridge: **import exercised 2026-07-22 in PenPot 2.17** with the multi-set shape now emitted natively as `penpot/tokens.penpot-import.json`; 71/74 tokens land (§9). Remaining: re-validate on future PenPot versions (the `duration` / `cubicBezier` drop and the `$description` quirk are 2.17 observations) and retire the operator-derived `../exports/penpot/tokens.penpot-import.json` in favour of the generated target.

- Compiler verification of the Kotlin / Swift / ArkTS / QML / Dart outputs in consumer-repo CI (kotlinc/swiftc/qmlint/dart were MISSING here — §5).

9. Verified consumer quirks (PenPot 2.17)

Both quirks were found during the **real PenPot 2.17 import on 2026-07-22** — not speculation.

1. **A top-level `$description` flips the tokens importer into single-set mode** [VERIFIED 2026-07-22]. Importing the then-`web/tokens.json` (which carried a root `$description` provenance header) made PenPot 2.17 ingest the file as **one set named “tokens” with 38 colors and no themes**, instead of the intended 3 sets + 2 themes. The importer evidently treats a root that contains any `$`-doc member as a single token group. The operator worked around it by hand-deriving `../exports/penpot/tokens.penpot-import.json` (sets + `$themes` + `$metadata`, no root `$description`) — which imported correctly. **Consequences baked into the generator:**
 - `web/tokens.json` stays the pure W3C DTCG canonical artifact (root `$description` header kept — harmless to DTCG tooling) and no longer claims to be the PenPot import file;
 - `generated/penpot/tokens.penpot-import.json` is emitted natively in the proven multi-set shape and is **byte-identical** to the operator-derived file (verified by `--check`, §4 item 5);
 - the penpot target is the **one generated file without a sha256 provenance header**: the only root members proven safe in 2.17 are the set names, `$themes` and `$metadata` — a `$comment` / `$description` root member is exactly the quirk-triggering shape, and `$metadata` extensions are unproven. Provenance is carried here and enforced by the `--check` byte-lock instead. The `$themes` UUIDs are pinned to the proven import’s ids so regeneration never mints new ones (determinism, §1).
 2. **`duration` and `cubicBezier` token types are silently dropped on import — 71/74 land** [VERIFIED 2026-07-22]. Affected: `--motion-fast` (duration), `--motion-base` (duration), `--ease-standard` (cubicBezier). **Choice: they are still INCLUDED** in `thready-structure` — mirroring the operator-derived file that proved the import, keeping the file lossless on the DTCG side and forward-compatible with a PenPot that learns these types. The generator prints a `penpot-unsupported:` note listing all three on every run so the 71/74 delta is never silent.
-

Made with love ♥ by Helix Development.